
K-Nearest Neighbors

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io
- Slides that adapt figures or content from other sources have a link to the source.

This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

Outline

- Overview
- 1–Nearest Neighbor
- K–Nearest Neighbor Classification and Regression
- Issues in Knn
- Variants of Knn
- Summary

Instance-Based Methods

- Instance-based methods use the training data instances directly to make predictions.
- Can be used for both classification and regression.
- Also known as nearest neighbor methods, lazy learners, memory-based methods.
- K-nearest neighbors (Knn) is an instance-based method.
- Called instance-based because of direct comparison with the training instance for predicting, rather than on generating an explicit hypothesis function.
- Similarity is measured using the features. The similarity function used will affect the hypothesis that is *learned*. We will see examples.

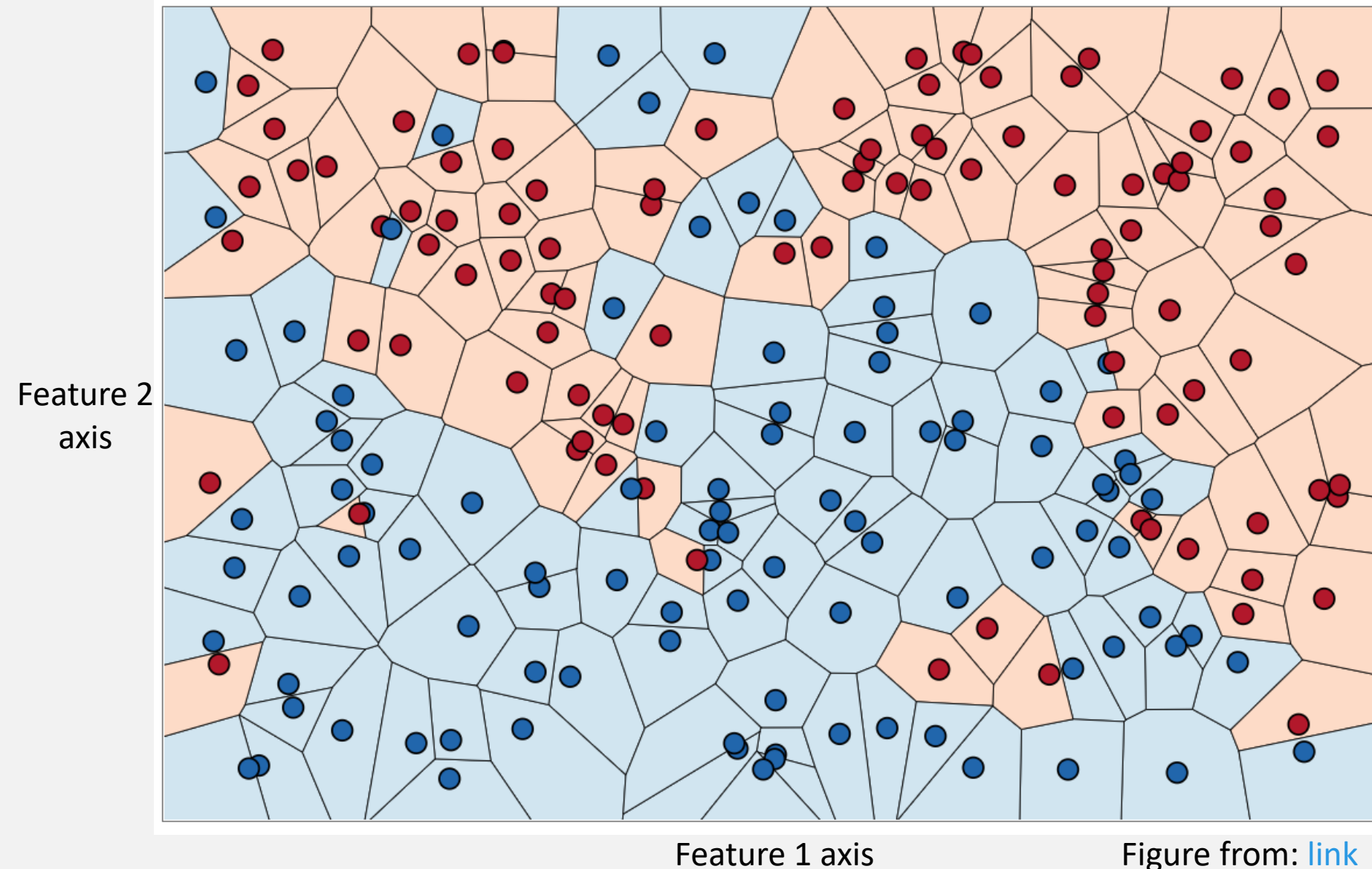
1-Nearest Neighbor (1NN)

- **Training Stage:** Given a training set $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$, do nothing (no training).
- **Prediction:** Given a new instance x_{new} :
 - Find the training instance $x_i \in T$ that is most similar to x_{new} .
 - Return the value of y_i of the most similar training instance as prediction. That is $y_{new} = y_i$
- Suppose $dist(\cdot, \cdot)$ is the distance measure we use, then the prediction can be explicitly expressed as:

$$(x_{nn}, y_{nn}) = \arg \min_{(x_i, y_i) \in T} dist(x_{new}, x_i) \quad \longleftarrow \text{1. Find the nearest neighbor (nn) training instance.}$$

$$\implies y_{new} := y_{nn} \quad \longleftarrow \text{2. Set the label of the new instance as the label of nearest training instance.}$$

1NN Decision Surface: Voronoi Diagram



- The red and blue points are the training data points of a binary classification task.
- With nearest neighbor methods, the decision surface is called a **Voronoi** diagram.
- Each Voronoi cell (polyhedron) indicates the region of each training instance.
- A new instance is classified according into which training instance region it falls into.

Figure from: [link](#)

K-Nearest Neighbor (KNN) Classification

- **Training Stage:** Given a training set $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$, do nothing (no training).
- **Prediction:** Given a new instance x_{new} :
 - Find the k training instances in T that are most similar to x_{new} .
 - Return the majority vote of the k most similar training instances.
- Let A be the k nearest neighbors of x_{new} , and let Y_A be the labels of these k nearest neighbors. Then prediction is as follows:

$$y_{new} := MajorityVote(Y_A) = \arg \max_{c \in \{1, 2, \dots, C\}} \text{sum}(Y_A == c)$$

Classification: Set the label of the new instance as the majority label of k-nearest training instances.

K-Nearest Neighbor (KNN) Regression

- **Prediction:** Given a new instance x_{new} :
 - Find the K training instances in T that are most similar to x_{new} .
 - Return the average label value of the K most similar training instances.
- Let A be the k nearest neighbors of x_{new} , then to predict:

$$y_{new} := \frac{1}{k} \sum_{(x_i, y_i) \in A} y_i$$

Regression: Set the label of the new instance as the average label of k-nearest training instances.

Knn Example [1]

- Given this training set T and x_{new} , use the euclidean distance and apply knn with $k = 3$. For simplicity, don't apply any rescaling of features.

length	width	height	weight	transport_type	Price (output)
109.66	83.53	112.01	947.33	0	3963.8
85.61	129.93	83.23	1434.64	0	3822.92
71.39	-0.67	68.04	1170.83	0	3999.95
100.11	29.88	13.48	787.55	0	3923.92
84.23	111.22	15.64	1442.46	0	4003.17
92.86	186.41	2.44	1113.22	1	4412.96
100.02	143.27	74.42	942.81	0	3918.71
90.3	91.86	25.53	927.77	1	3596.36
88.85	70.37	1.26	1081.61	1	3842.88

- $x_{new} = [95.24, 98.97, 84.17, 683.69, 1]$ what is the price prediction?

Knn Example [2]

- $x_{new} = [95.24, 98.97, 84.17, 683.69, 1]$

$$euclidean\text{dist}(x_i, x_j) = \sqrt{\sum_{q=1}^n (x_{iq} - x_{jq})^2}$$

$$\begin{aligned} euclidean\text{dist}(x_1, x_{new}) &= \sqrt{(109.66 - 95.24)^2 + (83.53 - 98.97)^2 + (112.01 - 84.17)^2 + (947.33 - 683.69)^2 + (0 - 1)^2} \\ &= 265.95 \end{aligned}$$

	length	width	height	weight	transport_type	Price (output)
(x_1, y_1)	109.66	83.53	112.01	947.33	0	3963.8
(x_2, y_2)	85.61	129.93	83.23	1434.64	0	3822.92
(x_3, y_3)	71.39	-0.67	68.04	1170.83	0	3999.95
(x_4, y_4)	100.11	29.88	13.48	787.55	0	3923.92
(x_5, y_5)	84.23	111.22	15.64	1442.46	0	4003.17
(x_6, y_6)	92.86	186.41	2.44	1113.22	1	4412.96
(x_7, y_7)	100.02	143.27	74.42	942.81	0	3918.71
(x_8, y_8)	90.3	91.86	25.53	927.77	1	3596.36
(x_9, y_9)	88.85	70.37	1.26	1081.61	1	3842.88

$euclidean(x_i, x_{new})$
265.95
751.65
498.06
143.46
762.04
445.9
263.11
251.17
407.52

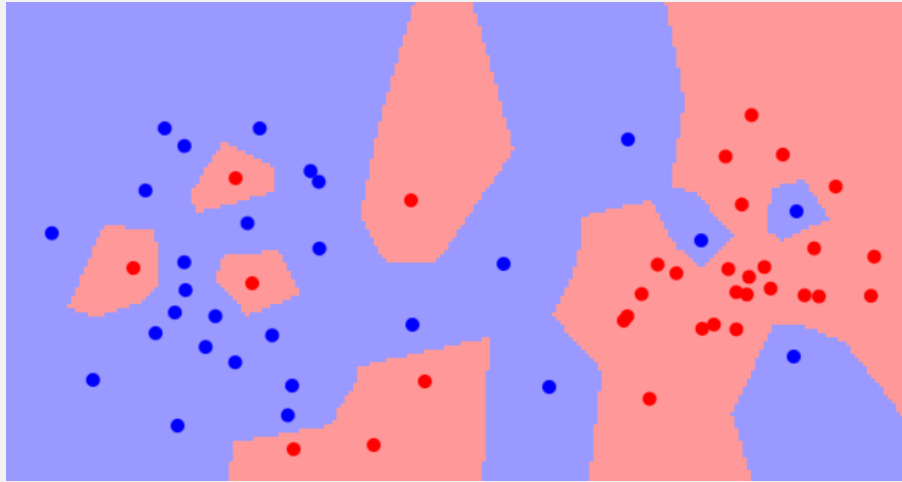
- Based on the distances, the 3-nearest neighbors are: $\{x_4, x_7, x_8\}$ so:

$$y_{new} = \frac{(y_4 + y_7 + y_8)}{3} = \frac{(3923.92 + 3918.71 + 3596.36)}{3} = 3813.0$$

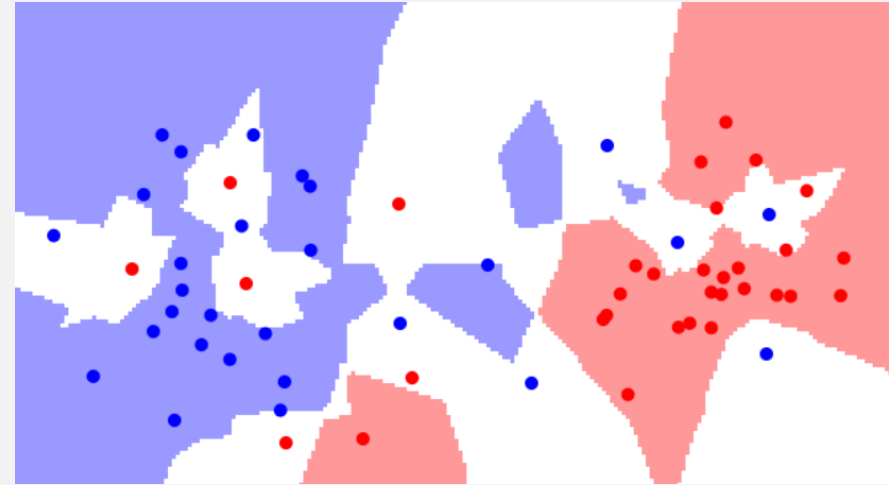
KNN Decision Surface: Voronoi Diagram – Binary classification on a 2-feature dataset

Feature 2 axis

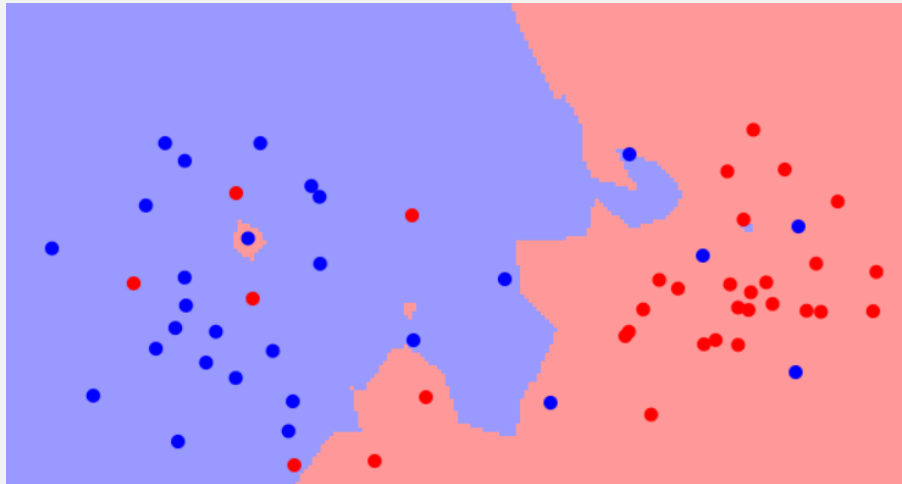
K = 1



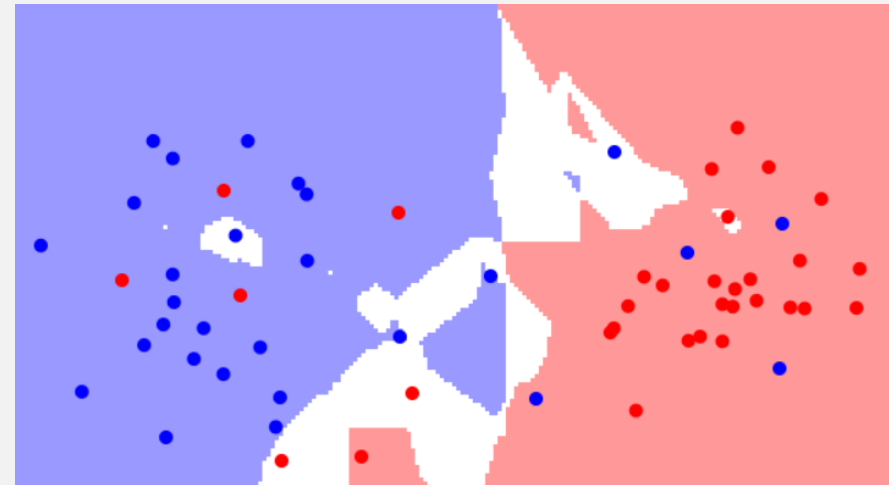
K = 2 (white areas indicate a tie)



K = 3



K = 4 (white areas indicate a tie)

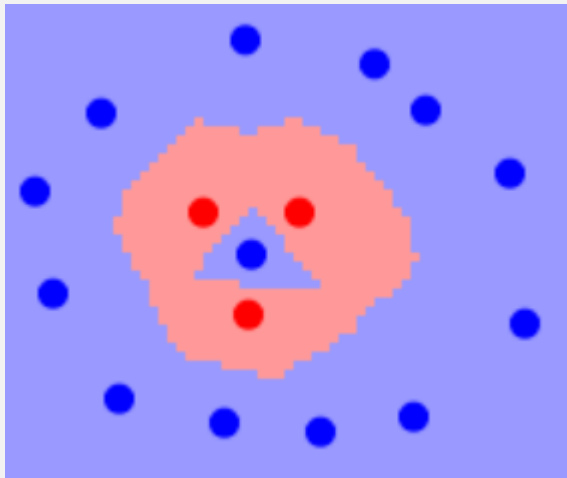


Feature 1 axis

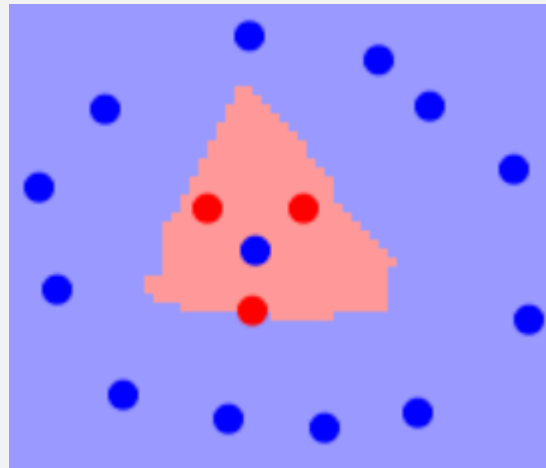
See knn demo: [link](#)

Effect of K value

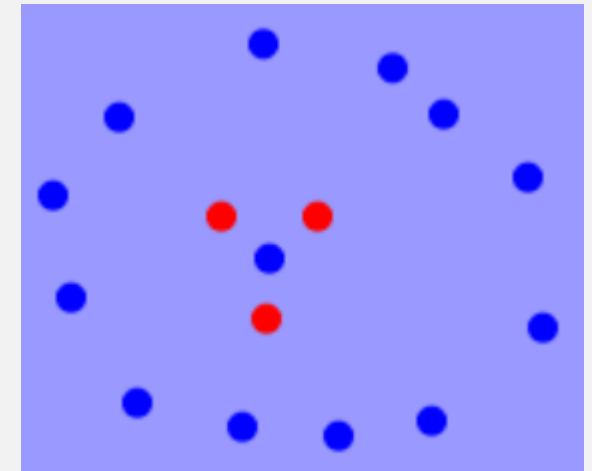
- If k is too small, then the prediction will be sensitive to noisy labels in the training data.
- If k is too large, may result in smoothening of important regional decision boundaries.
- How to select best value of K ? Use model-selection technique.



$k = 1$



$k = 3$



$k = 7$

Setting k -nn hyperparameter using K-Fold CV

- Note that the k in k -nn is a hyperparameter that you set.
- Can do K-fold Cross-Validation scheme to select this. Below we use 5-fold CV to select among 1-nn, 2-nn, and 3-nn.

Iteration	Train Set	Test Set	Test performance on 1nn	Test performance on 2nn	Test performance on 3nn
1	S_2, S_3, S_4, S_5	S_1	11/20 correct	10/20 correct	19/20 correct
2	S_1, S_3, S_4, S_5	S_2	17/20 correct	9/20 correct	18/20 correct
3	S_1, S_2, S_4, S_5	S_3	16/20 correct	11/20 correct	17/20 correct
4	S_1, S_2, S_3, S_5	S_4	13/20 correct	8/20 correct	18/20 correct
5	S_1, S_2, S_3, S_4	S_5	16/20 correct	19/20 correct	17/20 correct

73%.

57%.

89%.

We select $k = 3$ as it gives best Kfold CV performance.

- In practice, can search over a wider range of K values.

Distance Measures [1]

- Distance measure between two instances x_i and x_j is computed by aggregating the distance between their n features.
- Many different measures exist for discrete and continuous features. The best choice depends on the meaning of the feature and the learning task.
- If ALL your n features are discrete (finite set of values), a common distance measure is to assign a distance of 0 if the feature values are the same and 1 otherwise. This is called the **hamming distance**.

$$\text{hammingdist}(x_i, x_j) = \sum_{q=1}^n \begin{cases} 0 & \text{if } x_{iq} = x_{jq} \\ 1 & \text{otherwise} \end{cases}$$

Distance Measures [2]

- If ALL your features are continuous, one can use the **Euclidean distance**. Sum the squared difference between the features.

$$euclidean\text{dist}(x_i, x_j) = \sqrt{\sum_{q=1}^n (x_{iq} - x_{jq})^2}$$

- Alternatively, you can use the **Manhattan distance**. Sum the absolute difference between the features.

$$manhattan\text{dist}(x_i, x_j) = \sum_{q=1}^n |x_{iq} - x_{jq}|$$

Distance Measures [3]

- Thus, for two instances x_i and x_j , one way to compute the distance is:

$$dist(x_i, x_j) = \sum_{q=1}^n \begin{cases} hammingdist(x_{iq}, x_{jq}) & \text{if } q \text{ is a discrete feature} \\ |x_{iq} - x_{jq}| & \text{if } q \text{ is a continuous feature} \end{cases}$$

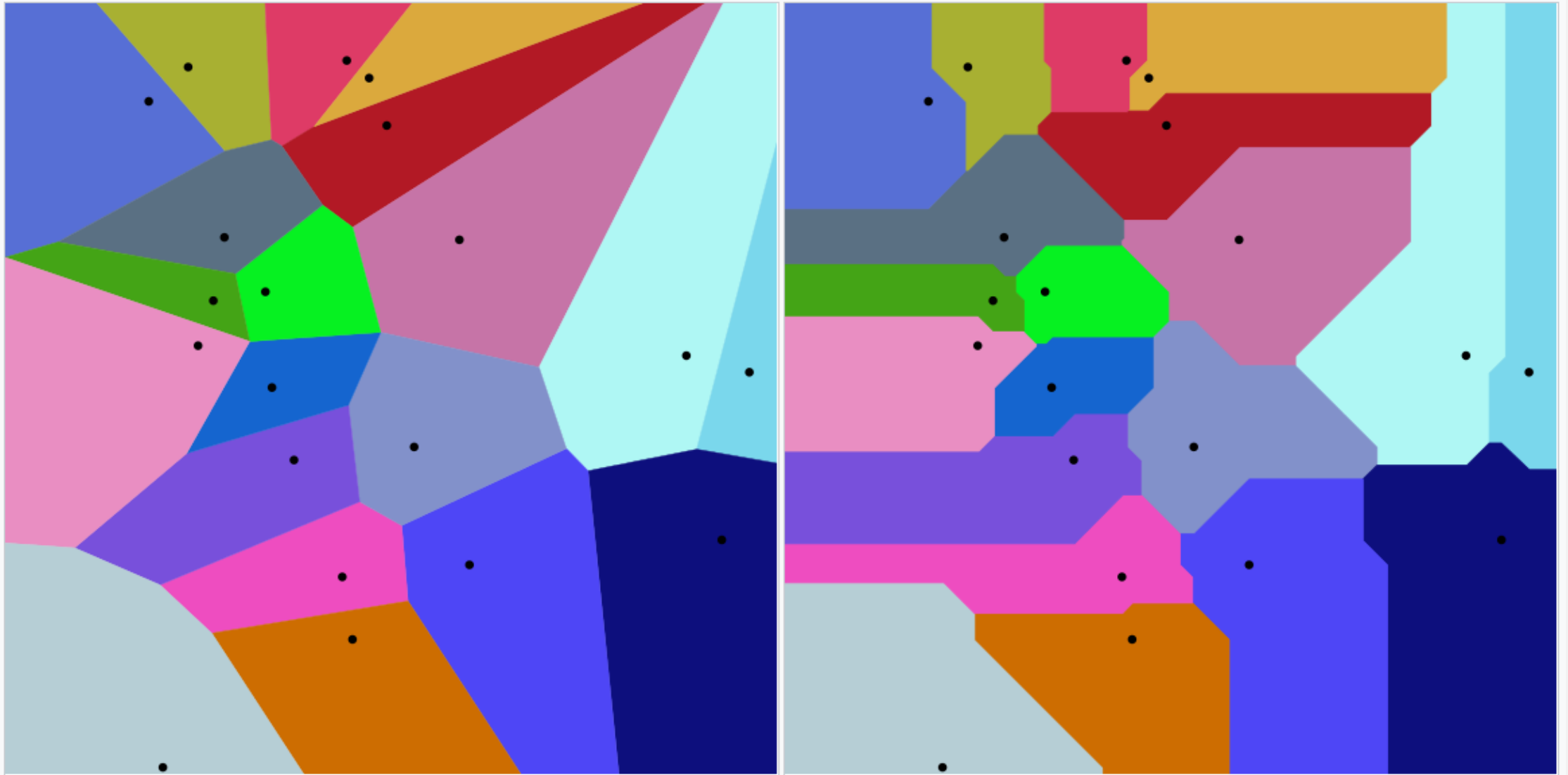
- Can also define a custom measure by explicitly defining different measures for each feature.
- **Note** that if you have converted your discrete string features into numbers using one-hot encoding, then you can just use euclidean or manhattan distance instead of hamming distance.

Distance Measures [4]

- Should we do feature scaling?
- Yes, because otherwise one feature with a large range might dominate the distance metric.
- Scale all continuous features to be in the $[0, 1]$ range (Min-Max Scaler).
Can also convert feature values to be standardized (Standard Scaler).

The choice of distance metric affects the resulting Voronoi decision surface

Voronoi diagrams of 20 points under two different metrics



Euclidean distance

Manhattan distance

Figure from: [link](#)

Knn Recap

- To use knn , you have to define the following:
 - Training dataset T .
 - Distance metric between two instances.
 - Number of nearest neighbors k . Can use Kfold CV or validation set to find good k .
- At test time (prediction), do the following:
 - Compute the distance of x_{new} to all training instances using the defined distance metric.
 - Select the k nearest training instances.
 - Compute the majority vote or average label value of the k nearest neighbors.

```
class sklearn.neighbors.KNeighborsClassifier(n_neighbors=5, *,  
weights='uniform', algorithm='auto', leaf_size=30, p=2,  
metric='minkowski', metric_params=None, n_jobs=None) #
```

```
class sklearn.neighbors.KNeighborsRegressor(n_neighbors=5, *,  
weights='uniform', algorithm='auto', leaf_size=30, p=2,  
metric='minkowski', metric_params=None, n_jobs=None) #
```

- See [KNeighborsClassifier](#) and [KNeighborsRegressor](#).
- Can use your own custom distance function.
- For most use-cases, you would just be concerned with n_neighbors and distance metric used.

Variants of Knn

Distance weighted nearest neighbors

- We can weigh nearest neighbor contributions according to their distance from x_{new} . Nearer instances contribute their labels more than farther instances.
- If A is the set of k nearest training instances to x_{new} , then:

Classification

$$y_{new} := \arg \max_{v \in \text{values}(Y)} \sum_{(x_i, y_i) \in A} w_i \delta(v, y_i) \longrightarrow$$
$$w_i = \frac{1}{\text{dist}(x_i, x_{new})}$$
$$\delta(a, b) = \begin{cases} 1 & \text{if } a = b \\ 0 & \text{otherwise} \end{cases}$$

Regression

$$y_{new} := \frac{\sum_{(x_i, y_i) \in A} w_i y_i}{\sum_{(x_i, y_i) \in A} w_i}$$

Denominator is to normalize the influence of each instance to be in the range $[0,1]$. Otherwise, a very near instance will have a large influence, whereas we just want the influence *relative to the other nearest neighbors*. Also, this same denominator can be added to classification formula above, but it will NOT change the argmax output.

Distance weighted Knn Example from [slide 18](#)

- $x_{new} = [95.24, 98.97, 84.17, 683.69, 1]$

	length	width	height	weight	transport_type	Price (output)
(x_1, y_1)	109.66	83.53	112.01	947.33	0	3963.8
(x_2, y_2)	85.61	129.93	83.23	1434.64	0	3822.92
(x_3, y_3)	71.39	-0.67	68.04	1170.83	0	3999.95
(x_4, y_4)	100.11	29.88	13.48	787.55	0	3923.92
(x_5, y_5)	84.23	111.22	15.64	1442.46	0	4003.17
(x_6, y_6)	92.86	186.41	2.44	1113.22	1	4412.96
(x_7, y_7)	100.02	143.27	74.42	942.81	0	3918.71
(x_8, y_8)	90.3	91.86	25.53	927.77	1	3596.36
(x_9, y_9)	88.85	70.37	1.26	1081.61	1	3842.88

$euclidean(x_i, x_{new})$
265.95
751.65
498.06
143.46
762.04
445.9
263.11
251.17
407.52

- Based on the distances, the 3-nearest neighbors are: $\{x_4, x_7, x_8\}$ so:

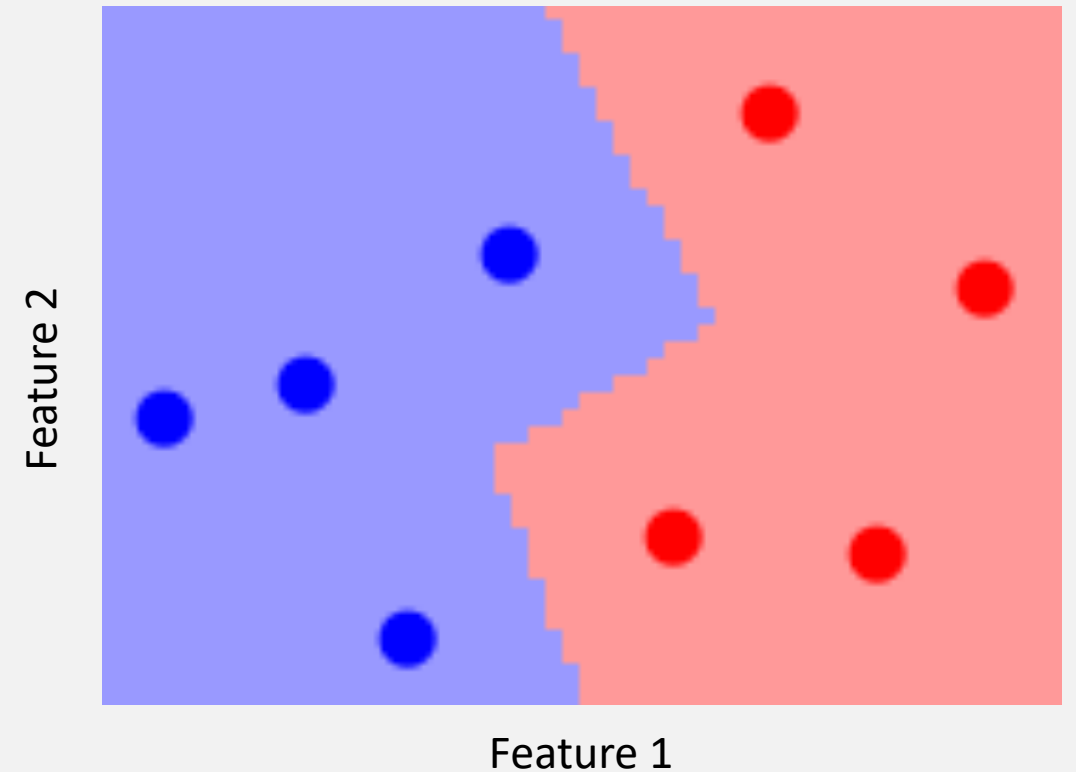
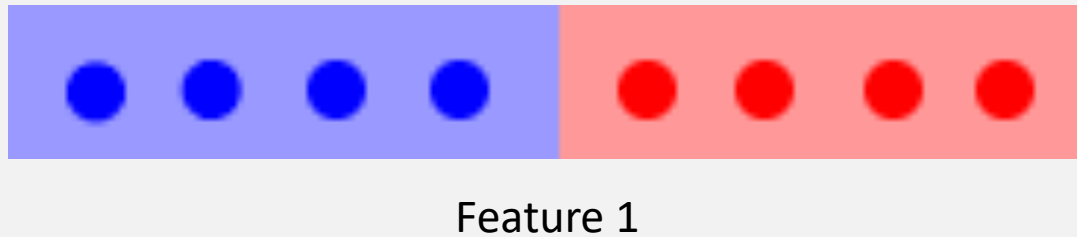
$$w_4 = \frac{1}{143.46} \quad w_7 = \frac{1}{263.11} \quad w_8 = \frac{1}{251.17}$$

$$y_{new} = \frac{w_4 y_4}{w_4 + w_7 + w_8} + \frac{w_7 y_7}{w_4 + w_7 + w_8} + \frac{w_8 y_8}{w_4 + w_7 + w_8} = 0.47y_4 + 0.26y_7 + 0.27y_8 = 3834.12$$

Note that the influence of each neighbor is relative and sum to 1.



Knn sensitivity to irrelevant features [1]



- In the left figure, there is only one feature, and a clear decision boundary. In the right figure, we introduce an irrelevant feature, that results in a less clear decision boundary between the two classes.

Knn sensitivity to irrelevant features [2]: locally weighted regression

- To reduce the effect of irrelevant features, one common solution is to weigh each feature differently.
- One such method is called locally weighted regression.
- To predict on x_{new} , get the k nearest training instances A . Then solve the following:

$$y_{new} = x_{new}^T w = w_0 + w_1 x_{new}^1 + w_2 x_{new}^2 + \cdots + w_n x_{new}^n$$

Where $w_0, w_1, \dots, w_n := \arg \max_{w \in \mathbb{R}^{n+1}} \sum_{(x_i, y_i) \in A} (x_i^T w - y_i)^2$

- That is, find the linear fit to the k nearest neighbors. Use the resulting feature weights to compute the prediction for x_{new} . Can be solved using gradient descent (we'll cover this later in the course).